

Run Managemob from *one panel* with Coolify

Coolify is a self-hosted platform that gathers deployment, environment variables, domains and monitoring in a single dashboard. This guide extends the original Managemob guide: your code, your AI assistant and Supabase stay exactly where they are — only the publication layer changes. You still don't write a single line of code by hand.

A GUIDE BY PESSOA ACADEMY · CONTINUES STEPS 01–10 OF "SHAPE MANAGEMOB IN YOUR OWN IMAGE"

4

NEW STEPS (11–14)

1

CONTROL PANEL

0

LINES OF CODE
WRITTEN BY HAND

~1 h

AVERAGE TIME

What Coolify is — and what it *replaces*

Coolify is a self-hosted deployment and application management platform designed to simplify how web applications are published, configured, updated and maintained. In the workflow you followed in the original guide, Managemob is developed locally in Visual Studio Code, synchronized through GitHub Desktop, connected to Supabase for data, and published on Netlify.

Adopting Coolify does not replace the application code, the AI-assisted editing workflow, or Supabase as the data layer. It replaces the publication layer centered on Netlify with a unified operational environment: deployment, runtime configuration, domains, environment variables and service monitoring, all managed from a single control panel.

WHY ADD COOLIFY

From a multi-tool chain to *one place*

The original guide relies on a multi-tool chain: GitHub stores the code, Visual Studio Code is the editing environment, an AI assistant writes the code, GitHub Desktop synchronizes changes, Supabase stores users and mobilities, and Netlify publishes the final site online.

This model is effective for getting started quickly, but it distributes responsibility across several interfaces. A person customizing Managemob must move between local development, repository synchronization, environment-variable handling, build settings, deployment configuration and domain management across separate tools. With Coolify, everything that happens after the push lives in one dashboard you own.

Scope of the new architecture

LAYER 01 • UNCHANGED

VS Code + AI assistant

Customization and code changes. You describe, the AI types — exactly as in steps 01–10.

LAYER 02 • UNCHANGED

GitHub repository

Version control and source management. GitHub Desktop keeps syncing your commits.

LAYER 03 • UNCHANGED

Supabase

Authentication, database and application data. Nothing moves, nothing is migrated.

LAYER 04 • NEW

Coolify

Deployment, runtime configuration, domains, environment variables and operational control — replacing Netlify.

Technical prerequisites

Before implementing Coolify, make sure the following conditions are met. They all correspond to steps you already completed in the original guide.

-
- 01 The Managemob repository (your fork) is already available on GitHub — step 02.
 - 02 Supabase is already configured: schema imported, authentication set up, admin user created — step 03.
 - 03 The application runs correctly in local development: `npm install` completed and `npm run dev` verified in the browser — step 05.
 - 04 The `.env` file remains excluded from version control through `.gitignore` — step 04.
 - 05 A server or virtual machine capable of hosting Coolify is available, since the organization chooses a self-hosted deployment model.
-

BONUS

Coolify itself is open source and free to self-host. The only cost is the server it runs on — a small cloud VM is enough to start.

Set up your *control room*

Why

Coolify is the operational layer that simplifies how Managemob is published and maintained. It gives the organization one place to connect the repository, define build behavior, store environment variables, assign domains and monitor the running application.

Actions

-
- 01 Prepare a server or cloud virtual machine where Coolify will run (Ubuntu LTS is the easiest choice).
 - 02 Install Coolify on that server according to the deployment model chosen by the organization, following the official installation script on coolify.io.
 - 03 Access the Coolify dashboard and create a project or workspace dedicated to Managemob.
 - 04 Verify that the server can reach both the Git provider and the Supabase endpoints used by the application.
-

PROMPT TO PASTE INTO THE AI ASSISTANT

"Explain what I should check on a fresh Ubuntu server before installing Coolify: open ports, firewall rules, DNS records pointing to the server IP. Give me a short checklist, don't change any file."

Same build, *new home*

Why

Coolify deploys directly from your GitHub fork, exactly as Netlify did. The application should preserve the same build assumptions already used in Netlify — so there is nothing to change in the code, only settings to mirror.

Actions

- 01 In your Coolify project, add a new application and connect it to your GitHub fork of Managemob (Coolify will ask for permission the first time, like Netlify did in step 09).
 - 02 Set the install step to use `package.json` dependencies (`npm install`).
 - 03 Set the build command to `npm run build`.
 - 04 Set the publish output to the `dist` folder (static site).
 - 05 Let Coolify handle runtime exposure through HTTP or its built-in reverse proxy — it manages SSL certificates automatically.
-

PROMPT TO PASTE INTO THE AI ASSISTANT

"Check the SPA routing for a static deployment on Coolify with nginx or its default proxy: what configuration do I need so that refreshing on a sub-page like /dashboard does not return a 404?"

ENVIRONMENT VARIABLES • 13

Move the *keys* into Coolify

Why

The original guide stores Supabase credentials in a local `.env` file during development and asks you to define the same variables again in Netlify during publication. With Coolify, production variables are managed in the application settings of the deployment environment rather than in Netlify. Your local `.env` keeps working for development — nothing changes on your computer.

Actions

- 01 In Coolify, open your Managemob application → Environment Variables.
 - 02 Add `VITE_SUPABASE_URL` with the Project URL you copied from Supabase in step 03.
 - 03 Add `VITE_SUPABASE_ANON_KEY` with the anon public key from the same page.
 - 04 Mark both as build-time variables: Vite reads them while building, not at runtime.
 - 05 Trigger a deploy and wait for the build to finish. Open the URL Coolify gives you and verify the login screen appears.
-

IMPORTANT

Never commit the `.env` file to GitHub and never paste the Supabase *service role* key into a front-end variable. Only the anon public key belongs in the browser — Row Level Security (step 03) is what protects your data.

DOMAIN & GO-LIVE · 14

Your address, your *rules*

Why

Right now the application answers on the URL generated by Coolify. To hand it to real users you connect your own domain and run the same final verification you did in step 10 — this time from the Coolify URL.

Actions

- 01 In your DNS provider, create an A record (e.g. `mobility.pessoa-academy.com`) pointing to the Coolify server IP.
 - 02 In Coolify, open the application → Domains → add the domain. Coolify issues the SSL certificate automatically.
 - 03 Enable automatic deployments: every push to your GitHub fork triggers a rebuild, exactly like Netlify did.
 - 04 Repeat the final checklist from the public domain.
-

Checklist

- ☐ Login on the public domain works.
 - ☐ The language switcher works in all 7 languages.
 - ☐ I add a participant and after F5 it's still there.
 - ☐ I generate a Word document with a participant: tags are filled in.
 - ☐ Refreshing /dashboard does not give a 404 error.
 - ☐ The SSL certificate is valid (padlock in the browser).
 - ☐ A push from GitHub Desktop triggers a new deploy in Coolify.
-

BEFORE OFFICIAL GO-LIVE

Re-enable email confirmation in Supabase Auth. Rotate the DB password. Invite the real users. Delete the test admin. Take a backup — and from now on, take it from the Coolify dashboard too.

CONCLUSION

One platform, one panel, *still zero code* written by hand

Visual Studio Code opens the files. The AI assistant writes the code. GitHub Desktop syncs. Supabase stores the data. Coolify now puts the site online and keeps it running — deployments, variables, domains and monitoring in a single place you own. You coordinate everything and make the decisions that matter. That's the Vibe Coding Workflow™, extended to operations.

VIBE CODING • PESSOA ACADEMY • MANAGEMOB • COOLIFY • SELF-HOSTED • GITHUB • SUPABASE •
ERASMUS+ • LEARNING MOBILITY